📘 Section 4: Training Engine & Optimization

📑 Table of Contents

1. Overview & Training Objectives

The training pipeline in `scripts/train_dual_stream_ddp.py` is designed to train the dual-stream neural network on large-scale datasets (such as 100,000+ face crops from FaceForensics++ and Celeb-DF v2) using multi-GPU hardware accelerators.

Training a dual-stream spatial-frequency network introduces unique optimization challenges: 1. **Optimization Asymmetry**: The spatial backbone has pre-trained weights, while the frequency stream is trained from scratch. 2. **Hardware Constraints**: 2D FFT operations and high-resolution crops (512×512) consume substantial GPU memory. 3. **Data Integrity**: Corrupted frames or read errors during distributed I/O must not pollute gradient updates.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              DDP TRAINING PIPELINE                           │
├──────────────────────────────────────────────────────────────────────────────┤
│ 1. Distributed Multi-GPU Setup (Hugging Face Accelerate + SyncBatchNorm)     │
│ 2. Deduplicated Split Ingestion & Class Balance Weighting (pos_weight)       │
│ 3. 4-Way Differential Optimizer (AdamW: Backbone=1e-4, Head=1e-3)            │
│ 4. Sequential Scheduler (Linear Warmup → Cosine Annealing)                   │
│ 5. Mixed Precision Forward Pass (FP16 Autocast with FP32 FFT Safety)         │
│ 6. Per-Sample Loss Masking (Filters Corrupted Frames via valid_flags)        │
│ 7. Gradient Accumulation (Micro-Batch 16 x 4 Steps = Effective Batch Size 64)│
│ 8. Exponential Moving Average Shadow Updates (beta = 0.999)                  │
│ 9. Distributed Metric Gathering & Validation ROC AUC Early Stopping          │
└──────────────────────────────────────────────────────────────────────────────┘
```

---

2. Distributed Data Parallel (DDP) Architecture

2.1 Single-GPU vs. Multi-GPU Distributed Training

- **Single-GPU Training**: Data is processed in sequential batches on a single device. When datasets exceed 100,000 images, training takes days.
- **Distributed Data Parallel (DDP)**: An identical copy of the model is instantiated across N discrete GPUs (e.g., Rank 0 on GPU 0, Rank 1 on GPU 1).
- Each GPU receives an independent subset of the training data.
- Forward passes execute in parallel across all GPUs.
- During the backward pass, an **All-Reduce** communication primitive averages the gradients across all GPUs before updating model weights.

2.2 Hugging Face `Accelerate` Integration

In `train_dual_stream_ddp.py#L212`, distributed training is managed using Hugging Face `Accelerate`:

```
accelerator = Accelerator(mixed_precision="fp16")
```

`Accelerate` automatically: 1. Detects the distributed environment (e.g., PyTorch `torchrun` / DDP). 2. Sets process ranks (`accelerator.is_main_process`). 3. Wraps models, optimizers, and data loaders via `accelerator.prepare(model, optimizer, train_loader, val_loader, scheduler)` . 4. Manages multi-GPU gradient reduction and synchronization.

2.3 Automatic Mixed Precision (AMP FP16)

Standard deep learning operations execute in 32-bit floating point (`fp32`). - **Mixed Precision (`fp16`)**: Executes matrix multiplications and convolutions in 16-bit float, reducing GPU VRAM usage by $\approx 50\%$ and doubling throughput on NVIDIA Tensor Cores. - **Dynamic Loss Scaling (`GradScaler`)**: Small gradient values in `fp16` risk underflowing to zero. `Accelerate` automatically multiplies the loss by a dynamic scale factor S before backpropagation, then un-scales gradients before the optimizer step:

$$\mathbf{g} = \frac{1}{S} \nabla_{\mathbf{w}} (S \cdot \mathcal{L})$$

---

3. Synchronized Batch Normalization (`SyncBatchNorm`)

Implemented in `scripts/train_dual_stream_ddp.py#L265` .

3.1 The Micro-Batch Statistical Problem

In standard `nn.BatchNorm2d` , running mean μ_B and running variance σ_B^2 are computed locally on the batch present on that specific GPU:

$$\mu_B = \frac{1}{M} \sum_{i=1}^{M} x_i, \quad \sigma_B^2 = \frac{1}{M} \sum_{i=1}^{M} (x_i - \mu_B)^2$$

- In multi-GPU training with a per-GPU micro-batch size of $M = 16$, computing statistics over only 16 samples produces high variance and noisy batch norm moments. - This severely impairs convergence in the frequency convolutional branch (`self.freq_conv`).

3.2 Cross-GPU Moment Aggregation

Before wrapping the model in DDP, `train_dual_stream_ddp.py` converts all BatchNorm layers into **Synchronized Batch Normalization**:

```
model = accelerator.sync_batch_norm(model)
```

`SyncBatchNorm` communicates across all GPU ranks during the forward pass:

$$\mu_{\text{global}} = \frac{1}{N \cdot M} \sum_{r=1}^{N} \sum_{i=1}^{M} x_{r,i}$$

This aggregates statistics across all GPUs ($16 \times 2 = 32$ samples), producing stable normalization moments across the distributed cluster.

---

4. Gradient Accumulation & Effective Batch Sizing

Implemented in `scripts/train_dual_stream_ddp.py#L303-L318`.

4.1 Why Large Batch Sizes Matter in Frequency Learning

Learning subtle steganographic high-pass noise patterns requires smooth, low-variance gradient updates. Training with small batch sizes ($B \leq 16$) creates erratic gradient directions that cause the gating mechanism to oscillate.

4.2 Simulating Batch Size 64 with Micro-Batches of 16

To achieve an effective batch size of 64 on hardware with limited VRAM:

$$\text{Effective Batch Size} = \text{Per-GPU Micro-Batch} \times \text{Number of GPUs} \times \text{Accumulation Steps}$$

$$\text{Effective Batch Size} = 16 \times 2 \times 2 = 64$$

In PyTorch, gradient accumulation accumulates gradients over $K = 4$ steps before calling `optimizer.step()`:

```
with accelerator.accumulate(model):
    optimizer.zero_grad(set_to_none=True)
    with accelerator.autocast():
        outputs = model(images)
        loss = (loss_unreduced * valid_flags).sum() / valid_flags.sum().clamp(min=1.0)
    accelerator.backward(loss)
    accelerator.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
```

4.3 Gradient Norm Clipping (`max_norm = 1.0`)

To prevent gradient explosions during backpropagation through the complex 2D FFT operations, gradient norms are clipped:

$$\mathbf{g}_{\text{clipped}} = \mathbf{g} \cdot \min\left(1.0,\ \frac{1.0}{|\mathbf{g}|_2}\right)$$

---

5. Loss Formulation & Per-Sample Loss Masking

5.1 Binary Cross-Entropy with Class Weighting (`pos_weight`)

In deepfake datasets, class distributions are often imbalanced (e.g., 5,639 Fake videos vs. 590 Real videos in Celeb-DF v2). In `train_dual_stream_ddp.py#L245-L252`, the positive class loss weight is dynamically computed from the training split:

$$w_{\text{pos}} = \frac{N_{\text{real}}}{\max(1, N_{\text{fake}})}$$

The weighted Binary Cross-Entropy loss for logit z_i and target label $y_i \in 0, 1$ is: $$\mathcal{L}_{\text{BCE}}(z_i, y_i) = -\Big[w_{\text{pos}} \cdot y_i \ln \sigma(z_i) + (1 - y_i) \ln(1 - \sigma(z_i)) \Big]$$

5.2 The Problem of Corrupted Image Frames

In large datasets with over 100,000 crops, occasional files may be corrupted on disk, truncated during download, or fail to decode in OpenCV. - A naive `DataLoader` either crashes or replaces corrupted images with all-black dummy arrays (0). - If an all-black dummy image is passed into the loss function with its original label, it injects severe label noise and corrupts the high-pass SRM filters.

5.3 Unreduced Masked Loss Computation

In `KaggleFastDataset.__getitem__`, when an image fails to load, `valid_flag = 0.0` is returned alongside a dummy tensor. If the image loads successfully, `valid_flag = 1.0`.

During training, loss is computed with `reduction='none'` and masked:

```
# 1. Compute unreduced loss for every sample in the batch [B, 1]
loss_unreduced = F.binary_cross_entropy_with_logits(
    outputs, labels, pos_weight=pos_weight_tensor, reduction="none"
)

# 2. Mask out invalid samples and average over valid samples only
loss = (loss_unreduced * valid_flags).sum() / valid_flags.sum().clamp(min=1.0)
```

$$\mathcal{L}_{\text{masked}} = \frac{\sum_{i=1}^B \mathcal{L}_{\text{BCE}}(z_i, y_i) \cdot v_i}{\max\left(1, \sum_{i=1}^B v_i\right)}$$ - Corrupted images ($v_i = 0$) contribute 0.0 **gradient** and do not impact model weights.

---

6. Differential Parameter Grouping & Optimization

Implemented in `get_differential_param_groups`.

6.1 The Transfer Learning Dilemma (Backbone vs. Frequency Branch)

- **Spatial Backbone (ConvNeXt-Small)**: Initialized with pre-trained ImageNet-1K weights. It requires a small learning rate ($\eta = 10^{-4}$) to fine-tune high-level semantic representations without destroying pre-trained weights.
- **Frequency Stream & Gating Head**: Initialized with random weights. They require a larger learning rate ($\eta = 10^{-3}$) to rapidly learn steganographic noise filters and gating projections from scratch.

6.2 4-Way Differential Parameter Grouping

`get_differential_param_groups` segments model parameters into 4 distinct optimization groups:

```
return [
    # Group 1: Backbone 2D Weight Tensors (Subject to Weight Decay)
    {"params": backbone_decay, "lr": 1e-4, "weight_decay": 1e-4},
    # Group 2: Backbone Biases and LayerNorms (No Weight Decay)
    {"params": backbone_nodecay, "lr": 1e-4, "weight_decay": 0.0},
    # Group 3: Head & Frequency 2D Weight Tensors (Subject to Weight Decay)
    {"params": head_decay, "lr": 1e-3, "weight_decay": 1e-4},
    # Group 4: Head & Frequency Biases, BatchNorms, Gating (No Weight Decay)
    {"params": head_nodecay, "lr": 1e-3, "weight_decay": 0.0},
]
```

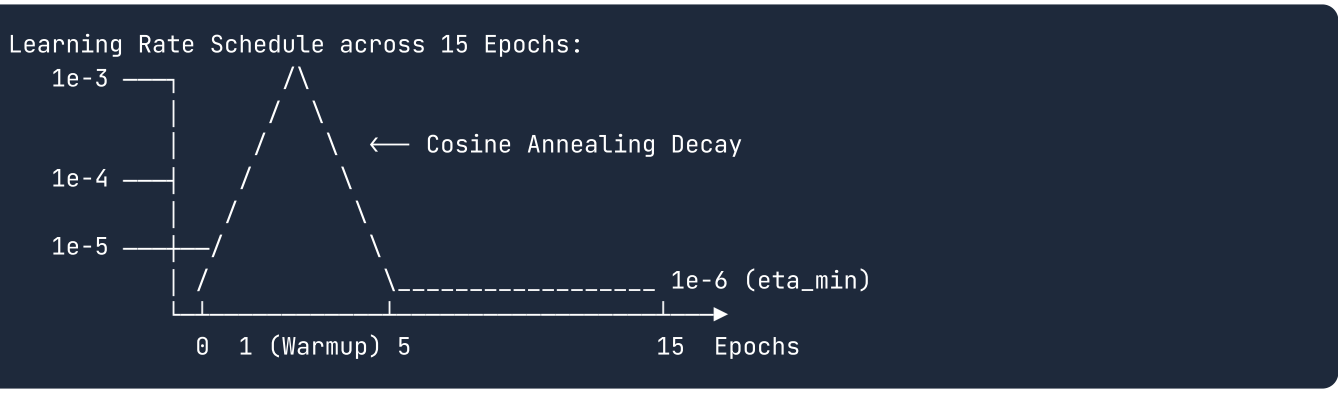
6.3 Weight Decay Exclusion for Biases and Normalizations

Applying L_2 weight decay to 1-dimensional tensors (biases b and LayerNorm/BatchNorm scale parameters γ, β) causes under-fitting and hurts calibration. - Weight decay ($\lambda = 10^{-4}$) is applied **strictly to 2D/4D weight matrices**. - Biases and normalization parameters receive $\lambda = 0.0$.

---

7. Learning Rate Scheduling: Warmup & Cosine Annealing

Implemented in `scripts/train_dual_stream_ddp.py#L255-L260`.



7.1 Phase 1: Linear Warmup (Epoch 1)

At the start of training, random initialization of the frequency branch can produce massive, erratic gradients that distort the pre-trained spatial backbone. - A **Linear Warmup Scheduler** (`LinearLR`) scales the learning rate from 10% of its target value up to 100% over the first epoch:

$$\eta(t) = \eta_{\text{base}} \cdot \left(0.1 + 0.9 \cdot \frac{t}{T_{\text{warmup}}}\right)$$

7.2 Phase 2: Cosine Annealing (Epochs 2–15)

After warmup, a **Cosine Annealing Scheduler** (`CosineAnnealingLR`) decays the learning rate following a cosine curve:

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\text{base}} - \eta_{\min}) \left(1 + \cos\left(\frac{t - T_{\text{warmup}}}{T_{\max} - T_{\text{warmup}}}\pi\right)\right)$$

where $\eta_{\min} = 10^{-6}$. This gradual decay allows the network to settle into narrow, high-generalization minima.

---

8. Exponential Moving Average (EMA) of Weights

Implemented in `ExponentialMovingAverage` .

8.1 Why EMA Produces Superior Forensic Checkpoints

Stochastic Gradient Descent (SGD/AdamW) causes model parameters to oscillate around optimal values from batch to batch. - Saving the model weights from the final training step captures a noisy point estimate. - **Exponential Moving Average (EMA)** maintains an averaged shadow copy of all model parameters across training iterations. EMA weights consistently yield higher validation AUC ($+0.5\%$ to $+1.2\%$) and significantly lower Expected Calibration Error.

8.2 Mathematical Shadow Parameter Update ($\beta = 0.999$)

On every training step, shadow weights θ_{EMA} are updated with decay factor $\beta = 0.999$:

$$\theta_{\text{EMA}}^{(t)} = \beta \cdot \theta_{\text{EMA}}^{(t-1)} + (1 - \beta) \cdot \theta_{\text{model}}^{(t)}$$

During validation: 1. `ema.apply_shadow(model)` swaps active model weights with shadow parameters. 2. Validation metrics are evaluated using the smooth shadow weights. 3. `ema.restore(model)` restores the active training parameters.

---

9. Validation, Early Stopping & Checkpoint Serialization

Implemented in `scripts/train_dual_stream_ddp.py#L326-L382` .

9.1 Multi-GPU Validation Metrics Gathering

During validation across multiple GPUs: 1. Each GPU computes predictions on its assigned validation chunk. 2. `accelerator.gather_for_metrics((preds, targets))` performs an all-gather communication step across all GPU processes. 3. Validation **Receiver Operating Characteristic Area Under Curve (ROC AUC)** is computed globally on Rank 0 using `sklearn.metrics.roc_auc_score` .

9.2 ROC AUC Early Stopping Mechanism

Early stopping monitors validation ROC AUC: - `EARLY_STOPPING_PATIENCE = 3` - If validation AUC fails to achieve a new best score for 3 consecutive epochs, training terminates early to prevent overfitting.

9.3 DDP Model Unwrapping & State Serialization

When saving the best checkpoint to disk:

```
unwrapped_model = accelerator.unwrap_model(model)
torch.save(unwrapped_model.state_dict(), BEST_MODEL_WEIGHTS_PATH)
```

- In DDP, layers are prefixed with `module.`.
- `accelerator.unwrap_model` strips distributed wrapper artifacts, saving a clean PyTorch `state_dict` that can be loaded seamlessly on any single GPU or CPU inference environment.

---

10. Code Walkthrough & Reference

Component / Function	File Reference	Primary Responsibility
`Accelerator` & DDP Setup	`train_dual_stream_ddp.py#L212-L269`	Initializes distributed training, SyncBatchNorm, and multi-GPU loaders.
`KaggleFastDataset`	`train_dual_stream_ddp.py#L122-L151`	Fast OpenCV image loader with `valid_flags` error trapping.
`get_differential_param_groups`	`train_dual_stream_ddp.py#L179-L205`	4-way optimizer parameter segmentation for differential learning rates.
`ExponentialMovingAverage`	`train_dual_stream_ddp.py#L69-L100`	Shadow parameter manager with $\beta = 0.999$ decay updates.
Training & Masked Loss Loop	`train_dual_stream_ddp.py#L290-L385`	Executes gradient accumulation, masked loss backpropagation, and validation AUC checkpoints.

This document serves as the permanent reference for Section 4 (Training Engine & Optimization) of the Deepfake Detection Engine.